

<https://app.notion.com/p/Go2-2d19a3e49dc88068a2f0e3f6cbbd3ba9#2d29a3e49dc88025b20fc32c6ada93bd>

本文是Jiabao Song所建立的 unitree go2 sim2sim/sim2real framework的实现流程。

This repository provides a framework for low-level control of a legged robot (Unitree Go2), using ROS 2 as the communication middleware. The MuJoCo sim

- **Ubuntu:** 20.04/22.04
- **ROS 2:** Foxy/Humble
- **MuJoCo:** 3.2.3
- **Python:** 3.8.20/3.10
- **Pinocchio:** 3.4.0

以下安装基于ubuntu 22.04、python版本3.10

Ubuntu (source) — ROS 2 Documentation: Humble documentation

You're reading the documentation for an older, but still supported, version of ROS 2.

For information on the latest version, please have a look at Kilted.

🔗 <https://docs.ros.org/en/humble/Installation/Alternatives/Ubuntu-Development-Setup.html>

```
sudo apt install ros-foxy-rmw-cyclonedds-cpp
sudo apt install ros-foxy-rosidl-generator-dds-idl
sudo apt install libyaml-cpp-dev
```

```
#source /home/zelin/miniconda3/etc/profile.d/conda.sh
source ~/miniconda3/etc/profile.d/conda.sh
#conda create -n go2_deploy_ros2_env python=3.8
conda create -n go2_deploy_ros2_env python=3.10
conda activate go2_deploy_ros2_env
```

1

- 主部署仓库: `Deploy-an-RL-policy-on-the-Unitree-Go2-robot`。

```
# 1. git the main repo
git clone https://github.com/Glowing-Torch/Deploy-an-RL-policy-on-the-Unitree-Go2-robot.git
```

<https://github.com/Glowing-Torch/Deploy-an-RL-policy-on-the-Unitree-Go2-robot>

- Unitree ROS2 SDK: 克隆到主仓库的 `src` 目录下。

```
# 2. git unitree ros2 sdk in main repo
cd src
#cd Deploy-an-RL-policy-on-the-Unitree-Go2-robot/src
git clone https://github.com/unitreerobotics/unitree_ros2.git
```

https://github.com/unitreerobotics/unitree_ros2

- CycloneDDS 相关 SDK: 克隆到 `unitree_ros2/cyclonedds_ws` 目录下。cd

```
# 3. git cyclonedds sdk in unitree_ros2
# ~/Deploy-an-RL-policy-on-the-Unitree-Go2-robot/s/unitree_ros2/cyclonedds_ws
cd unitree_ros2/cyclonedds_ws
git clone https://github.com/ros2/rmw_cyclonedds -b foxy # humble if using ROS2 humble
git clone https://github.com/eclipse-cyclonedds/cyclonedds -b releases/0.10.x
```

2、分层编译

注意: 在执行编译命令前, 必须先退出 (deactivate) Conda 环境, 否则可能导致路径冲突。

项目使用 `colcon build` 工具进行分段编译:

- 编译 CycloneDDS: 在 `cyclonedds_ws` 目录下单独编译该包。

如果遇到错误删除编译文件: `rm -rf build install log`

```
# pwd = ../go2_deploy/src/unitree_ros2/cyclonedds_ws
colcon build --packages-select cyclonedds
# output
Starting >>> cyclonedds
Finished <<< cyclonedds [8.49s]Summary: 1 package finished [9.67s]
```

如果遇到报错:

```
~/Deploy-an-RL-policy-on-the-Unitree-Go2-robot/s/unitree_ros2/cyclonedds_ws on master ?2
> colcon build --packages-select cyclonedds
Starting >>> cyclonedds
--- stderr: cyclonedds
../../../../bin/idlc: symbol lookup error: ../../../../bin/idlc: undefined symbol: DDS_XTypes_TypeObject_desc
make[2]: *** [src/tools/ddsp perf/CMakeFiles/ddsp erf_types_generate.dir/build.make:63: src/tools/ddsp erf/ddsp erf_types.c] Error 127
make[1]: *** [CMakeFiles/Makefile2:573: src/tools/ddsp erf/CMakeFiles/ddsp erf_types_generate.dir/all] Error 2
make: *** [Makefile:152: all] Error 2
```

```

---
Failed    <<< cyclonedds [15.3s, exited with code 2]

Summary: 0 packages finished [15.6s]
  1 package failed: cyclonedds
  1 package had stderr output: cyclonedd

```

这是unitree_ros2自带的问题，在readme中有解决方法：

```

rm -rf build install log
export LD_LIBRARY_PATH=/opt/ros/foxy/lib
colcon build --packages-select cyclonedds

```

- **编译 Unitree_ROS2:** 回到上一级目录编译整个 SDK。如果 `unitree_ros2_example` 报错可以忽略。

```

    cd source /opt/ros/humble/setup.zsh
# pwd = ../go2_deploy/src/unitree_ros2
colcon build
# output
Starting >>> cycloneddsStarting >>> unitree_apiStarting >>> unitree_goStarting >>> unitree
_hg
Finished <<< unitree_api [8.71s]Finished <<< cyclonedds [10.3s]
Starting >>> rmw_cyclonedds_cpp
Finished <<< unitree_hg [10.7s]
Finished <<< rmw_cyclonedds_cpp [4.45s]
Finished <<< unitree_go [16.2s]
Starting >>> unitree_ros2_example---
stderr: unitree_ros2_example
...
---
Finished <<< unitree_ros2_example [14.1s]
Summary: 6 packages finished [30.4s]
1 package had stderr output: unitree_ros2_example
# error of unitree_ros2_example is not matter

```

- **编译 RL 策略包:** 在项目根目录下编译 `deploy_rl_policy`

编译前要执行 `source ../go2_deploy/src/unitree_ros2/install/setup.zsh (.bash)` ,

```

cd ..
# pwd = ../go2_deploy
#source ~/Deploy-an-RL-policy-on-the-Unitree-Go2-robot/src/unitree_ros2/install/setup.zsh
source ../go2_deploy/src/unitree_ros2/install/setup.zsh
colcon build --packages-select deploy_rl_policy
# output
G02 Deploy 3Starting >>> deploy_rl_policy
Finished <<< deploy_rl_policy [8.62s]
Summary: 1 package finished [8.75s]

```

仿真运行

编译完成后，可以先在 MuJoCo 物理引擎中进行仿真测试。

- **环境设置:** 重新激活 Conda 环境，安装 `torch` 和 `mujoco`。（自动下载合适的版本就ok）

```

conda activate go2_deploy_ros2_env
pip install torch

```

```
pip install mujoco
```

- **环境变量加载**：需要依次 `source` 系统 ROS2 环境、SDK 安装环境以及项目安装环境。（推荐使用direnv加载）

```
source ~/miniconda3/etc/profile.d/conda.sh
conda activate go2_deploy_ros2_env
source /opt/ros/humble/setup.bash
source ../go2_deploy/src/unitree_ros2/install/setup.bash
source ../go2_deploy/install/setup.bash
#在命令行中使用zsh，在.envrc中使用.bash
```

▼ direnv使用方法

direnv 的基本加载逻辑

`direnv` 的核心原理是 挂钩（Hook）。当你进入一个包含 `.envrc` 文件的目录时，它会自动执行以下流程：

1. 检测目录：Shell 插件检测到进入了新目录。
2. 查找文件：搜索是否存在 `.envrc` 文件。
3. 安全检查：如果文件内容变动或首次运行，会要求你执行 `direnv allow`。
4. 导出变量：`direnv` 会启动一个子 shell 执行该脚本，捕获所有环境变量的变化，并将其应用到你当前的 Shell 中。
5. 自动卸载：一旦你 `cd ..` 离开该目录，它会自动撤销这些变量，保持系统的干净。

配置流程

如果你还没有配置好 `direnv`，请按照以下步骤操作：

第一步：安装并配置 Shell

```
# 安装
sudo apt install direnv
# 打开 .zshrc
gedit ~/.zshrc

# 在你的 ~/.zshrc 最后一行添加挂钩
eval "$(direnv hook zsh)"
```

完成后记得重启终端或执行 `source ~/.zshrc`。

第二步：创建并授权

1. 进入项目根目录：`cd ../go2_deploy`。
2. 创建文件：`.envrc`（粘贴上面的代码并保存）。
3. 授权加载：

```
direnv allow .bashrc
```

- **启动节点**：在**不同的终端**中运行以下四个节点：

1. **MuJoCo 仿真器**：负责物理模拟。

```
ros2 run deploy_rl_policy mujoco_simulator.py
```

2. **Xbox 控制器接口**：接收手柄指令。

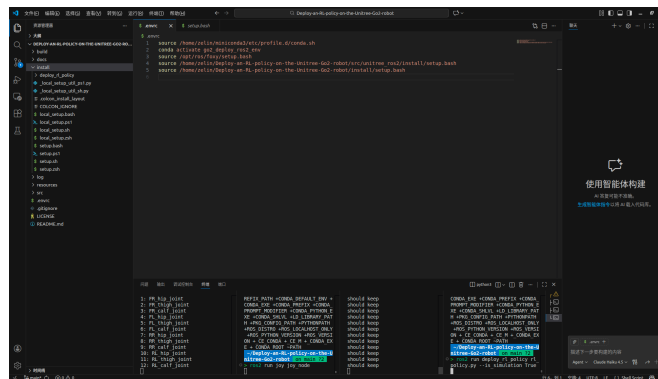
```
ros2 run joy joy_node
```

3. 状态机控制器：管理机器人动作状态（需开启 `is_simulation` 模式）。

```
ros2 run deploy_rl_policy low_level_ctrl --ros-args -p is_simulation:=true
```

4. RL 策略节点：运行强化学习算法推理。

```
ros2 run deploy_rl_policy rl_policy.py --is_simulation True
```



手柄操作说明

在仿真中，可以使用 Xbox 手柄控制机器人的行为：

- 初始状态：机器人自动进入“趴下”状态。
- B 键：从“趴下”切换为“站立”。
- A 键：从“站立”切回“趴下”。
- LB + RB 同时按下：激活 RL 策略模式。保持按住这两个键，即可通过摇杆自由控制机器人移动。

与Unitree go2通信

1. 将go2作为主机，外接显示屏

将TypeC转HDMI转接头插入Type-C全功能接口，使用HDMI线连接显示屏。

连接显示屏后可以直接进入go2的Ubuntu系统20.04

```
username: unitree password:123
```

```
IP: 192.168.123.18
```

2、使用网线连接go2通信

1. 将自己的电脑通过网线与go2拓展坞的RJ45接口连接
2. 在ubuntu内，设置-网络-有线的设置中，手动设置电脑的静态ip

SSH登陆端口IP地址必须为：192.168.123.xxx
子网掩码：255.255.255.0

3. 下一步验证与go2是否正常通信：（ping通后就可以直接在主机上跑node进行控制）

```
ping 192.168.123.18 #拓展坞PC IP
ping 192.168.123.20 #雷达
ping 192.168.123.161 #运控PC IP
```

4. 远程登录拓展坞PC，在命令行中远程登录,ROS版本选择 `ROS2 foxy`

```
ssh unitree@192.168.123.18
Passcode: 123
```

5. 如果想使用机载ubuntu系统的图形界面，可以使用no machine工具

```
https://download.nomachine.com/download/?id=1&platform=linux
```

使用no machine远程连接时，可能会出现可以显示桌面但无法操作的现象，此时需要将go2的万用接口插上拓展坞连接HDMI线才能远程操作。

3、将文件传入unitree主机

1. 使用scp指令传输：

```
exit#退出unitree@ubuntu
scp -r ./本地文件夹路径 unitree@192.168.123.18: /home（目标路径）
passcode: 123
```

2、使用VS Code 插件进行传输：Remote-SSH

- 安装Remote-SSH
- Ctrl + Shift + P，输入Remote-SSH: Connect to Host
- 选择或输入：unitree@192.168.123.18 passcode: 123
- File - Open Folder - /home
- 将目标文件拖入即可

实机部署

在进行go2实机部署前，应该首先关闭机器人自带的sport_mode，防止自己的policy与其产生冲突。

- 下载unitree go app，登陆授权账号。
- 开机unitree go2，等待其预加载完成后使用手机wifi连接“家人”wifi，获得go2的控制权。
- 打开shebei fuwuzhuangtai，关闭sport_mode

实机部署的几种方法：

- 连接主机进行部署（需要连接网线）：
 1. 按照与go2通信模块的步骤，走到ping验证通信是否成功这一步
 2. 在解除sport_mode的前提下，在主机上运行各node，然后使用手柄操作即可。
- 在机载Jetson计算平台上进行部署（部署完成后不需要外接线路）：
 1. 首先将git好的文件库（注意层级关系），传入go2自带的ubuntu 20.04。
 2. 在jetson中完成项目的分层级build，并运行各节点。
 3. 拔掉显示屏，使用手柄操作go2机器狗。